DAMG 7370

Designing Advanced Data Architectures
for Business Intelligence

Midterm Team Project

Data Engineering Pipeline — Chinook Dataset

Databricks | Unity Catalog | DQX | Job Orchestration

Gaurav Dalal · Pratik Kanade · Michael Dereje

Northeastern University

Spring 2026

1. Introduction

This report documents the design and implementation of a metadata-driven data engineering pipeline built on the Databricks platform.

The Chinook dataset models a digital media store, containing information about artists, albums, tracks, invoices, customers, and employees. It includes 11 interconnected tables that represent a typical transactional system one might encounter in industry. For this project, two versions of the dataset were provided — Version 1 for the initial full load and Version 2 to demonstrate how the pipeline handles incremental changes, particularly for Slowly Changing Dimensions.

Our pipeline follows the Medallion Architecture pattern (Raw → Bronze → Silver → Gold), where each layer serves a specific purpose in progressively refining the data from its raw source form into a clean, analytics-ready dimensional model. The entire pipeline is orchestrated through a Databricks Job, with each stage implemented as an individual parameterized notebook.

The key objectives were to build a fully parameterized pipeline with no hardcoded values, implement metadata-driven ingestion so that adding a new source table requires only a metadata row and zero code changes, enforce data quality validation before data reaches the Silver layer, implement SCD Type 2 tracking for the Customer dimension, and tie everything together with end-to-end job orchestration.

The sections that follow walk through the architecture, the implementation of each pipeline stage, the data quality framework, the Gold layer dimensional model, and the job orchestration setup. We also cover the challenges we ran into and how we resolved them.

2. Architecture Overview

The pipeline is structured around the Medallion Architecture, which organizes data into four progressively refined layers. Each layer has a distinct storage format, write mode, and purpose. The general idea is that Raw captures the source exactly as-is, Bronze makes it queryable in Delta format, Silver cleans and validates it, and Gold shapes it into a dimensional model ready for analytics.

The table below summarizes the four zones:

Zone	Storage	Format	Write Mode	Purpose
Raw	Databricks Volume	Parquet	Append (snapshot)	Immutable copy of source data. Each run adds a new timestamped snapshot.
Bronze	Unity Catalog	Delta	Overwrite	Current-day Delta snapshot. If downstream breaks, reload from Raw.
Silver	Unity Catalog	Delta	Overwrite	Cleaned and quality-validated data. DQX checks act as a gate.
Gold	Unity Catalog	Delta	Merge / Overwrite	Star schema with surrogate keys, SCD Type 2, and aggregated facts.

Data flows through the pipeline as: Source (Azure SQL) → Raw (Parquet in Volume) → Bronze (Delta) → DQX Validation → Silver (Delta) → Gold (Delta). This is a linear dependency chain, meaning each stage only runs after the previous one succeeds.

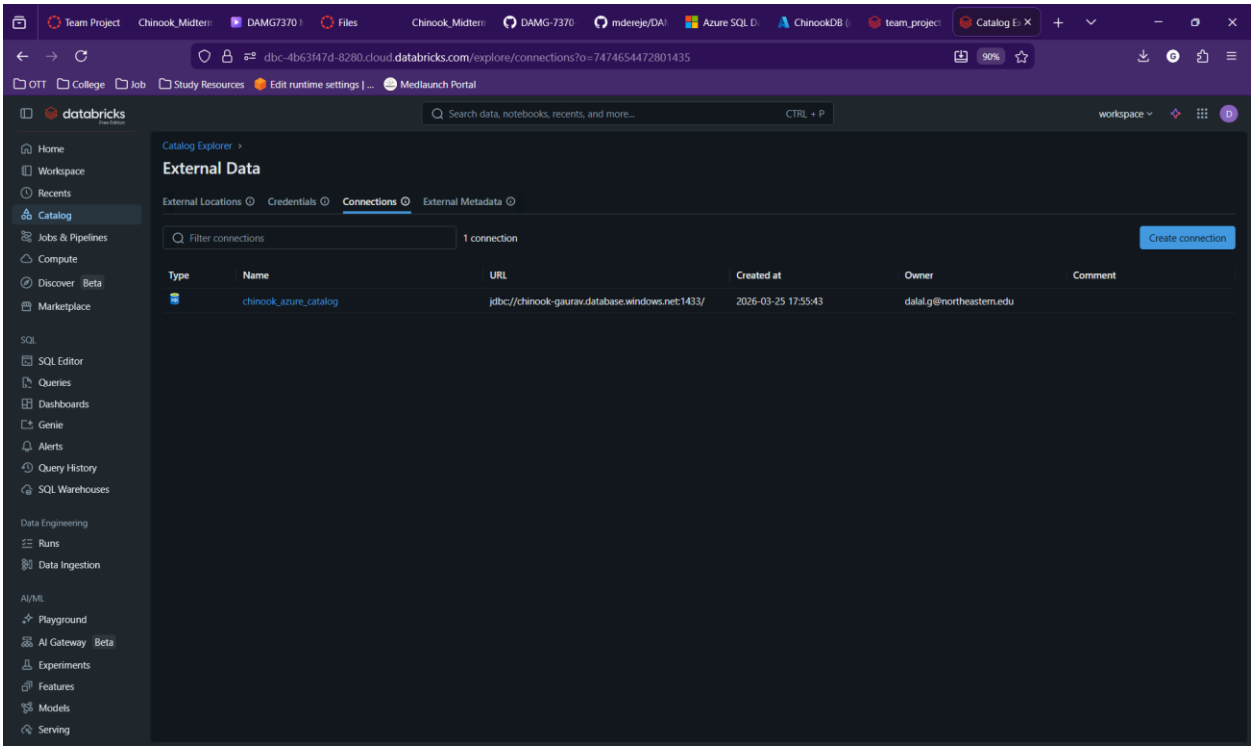
All source connectivity is handled through the Databricks Connection Manager, which federates access to an Azure SQL database hosting the Chinook schema. No connection strings or credentials appear anywhere in the notebooks — tables are referenced using federation syntax like azure_sql_chinook_catalog.chinook.Album.

Unity Catalog provides governance across all layers. Schemas for raw_zone, bronze, silver, and gold are created under the workspace catalog, and every table is registered there so that it benefits from access controls, lineage tracking, and discoverability.

3. Environment Setup

3.1 Connection Manager

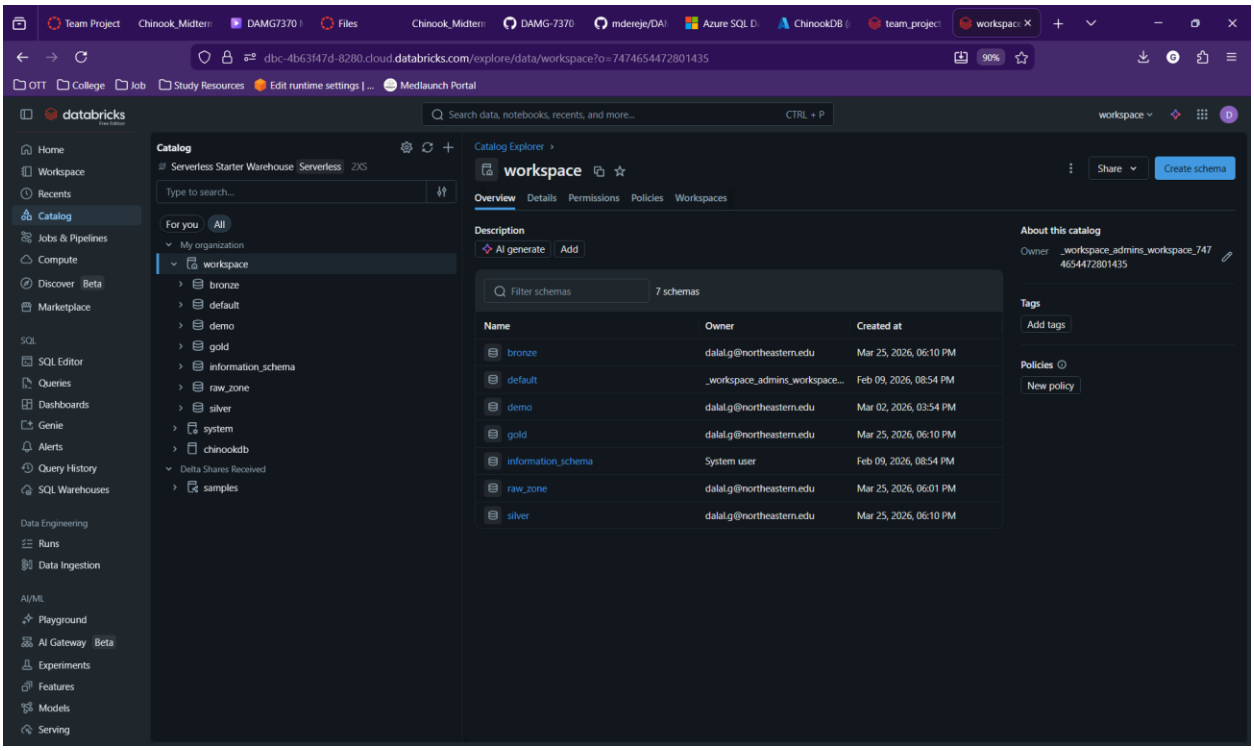
We configured the Databricks Connection Manager to point to the Azure SQL database hosting the Chinook dataset. The connection was created through the Catalog UI under Settings → Connections. Once established, all notebooks reference source tables using federation syntax (e.g., azure_sql_chinook_catalog.chinook.Customer) without embedding any credentials. This is a hard requirement from the project spec and mirrors how production pipelines typically handle source connectivity.



3.2 Volume and Schema Creation

The Raw zone stores Parquet files inside a Unity Catalog Volume. We created the volume at workspace.raw_zone.chinook using a sql command. The Volume path (/Volumes/workspace/raw_zone/chinook/) serves as the base path for all raw file writes.

We also created four schemas under the workspace catalog: raw_zone, bronze, silver, and gold. The 00_setup.py notebook handles all of this automatically — it loops through the schema names and runs CREATE SCHEMA IF NOT EXISTS for each one. This makes the environment reproducible so any team member can run the setup once and be ready to go.



3.3 Notebook Parameterization

Every notebook in the pipeline reads its configuration through Databricks widgets using dbutils.widgets.get(). None of the notebooks contain hardcoded catalog names, schema names, or file paths. The parameters are passed in through the Databricks Job configuration, which means the same notebooks could point to a different catalog or schema without any code changes.

The key parameters used across the pipeline include:

Parameter	Description	Example
catalog_name / destination_catalog	Unity Catalog name	workspace
schema_name / destination_schema	Target schema within the catalog	raw_zone
base_path	Volume base path for raw Parquet files	/Volumes/workspace/raw_zone/chinook
source_catalog	Federated catalog for the Azure SQL source	azure_sql_chinook_catalog
source_schema	Schema within the source catalog	chinook

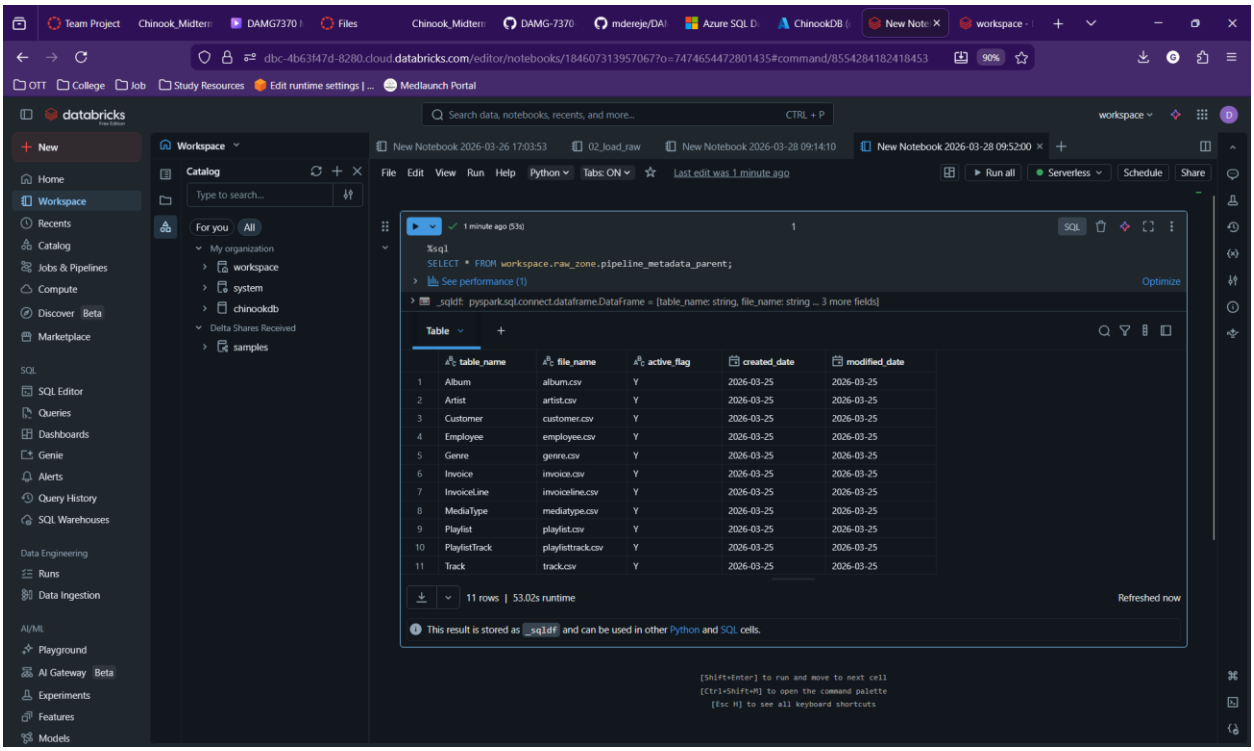
4. Metadata-Driven Pipeline Framework

One of the core requirements is that the pipeline must be metadata-driven. In practical terms, this means no table names are hardcoded in any notebook. Instead, the pipeline reads from two metadata tables to determine what to process and to record what happened during each run.

4.1 Parent Metadata Table

The parent metadata table (pipeline_metadata_parent in the raw_zone schema) contains one row per source table. Each row has the table name, the corresponding file name, and an active_flag column. When the pipeline runs, it queries this table and only processes rows where active_flag = 'Y'. If we ever need to add a new table, we just insert a row — no code changes needed.

The 00_setup.py notebook seeds this table with all 11 Chinook source tables (Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLine, MediaType, Playlist, PlaylistTrack, and Track). It checks for existing entries first to avoid duplicates on re-runs.

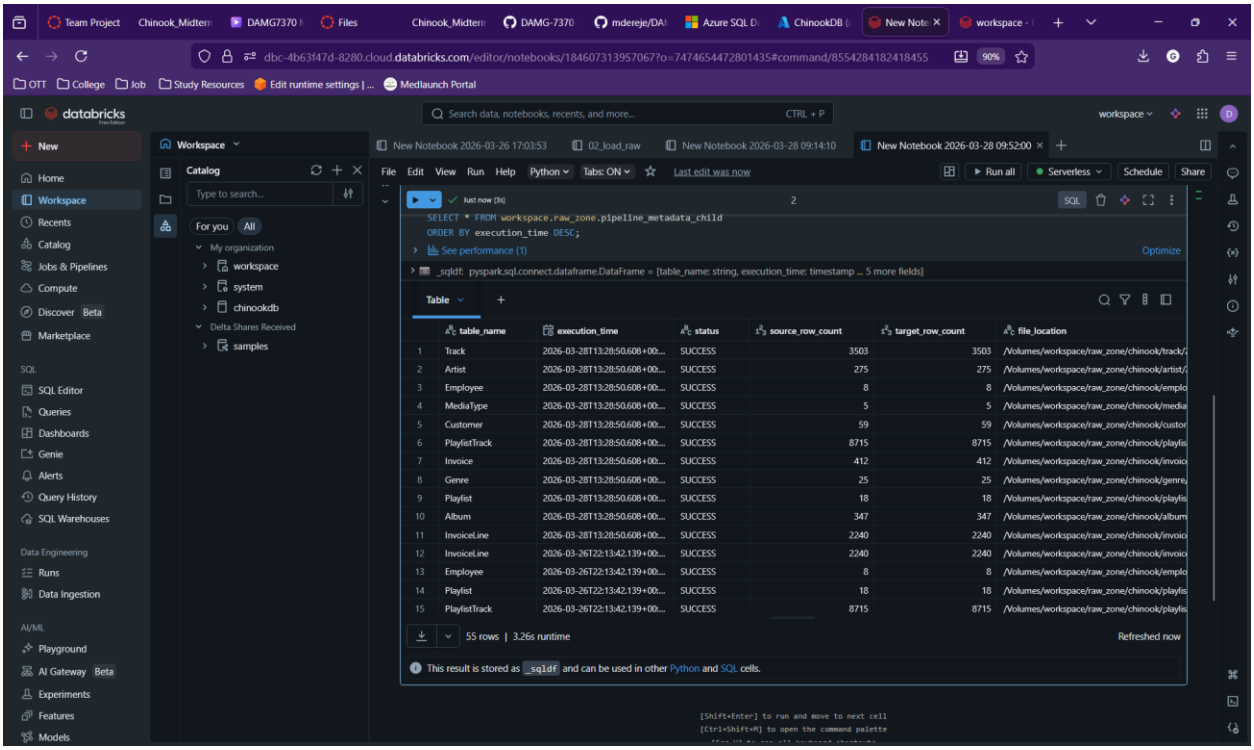


4.2 Child Execution Metrics Table

The child execution metrics table (pipeline_metadata_child in raw_zone) logs one row per table per pipeline run. Each row captures the table name, execution timestamp, status (SUCCESS or FAILED), source row count, target row count, and the full file path of the Parquet snapshot that was written.

The source and target row counts are compared during the Raw load step. If they do not match, the status is set to FAILED and the pipeline raises an exception. This count validation guarantees that the Raw zone

is an exact copy of the source. Downstream stages also use this table to find the latest successful file location for each table, so the metadata tables effectively drive the pipeline's entire data flow.



5. Pipeline Implementation

5.1 Source Connectivity Validation (01_extract_from_source)

The first notebook is a connectivity check. It reads the parent metadata table to get the list of active tables, then attempts to query each corresponding source table through the Connection Manager. For every table, it runs a spark.read.table() with a limit(1) just to confirm the source is reachable.

If any table fails the connectivity check, the notebook raises an exception immediately with the list of unreachable tables. This fail-fast approach prevents the pipeline from wasting time on downstream steps when the source is unavailable. It is worth noting that this notebook does not pass data downstream — each subsequent notebook reads from the source independently, since temporary views do not persist across notebook boundaries in a Databricks Job.

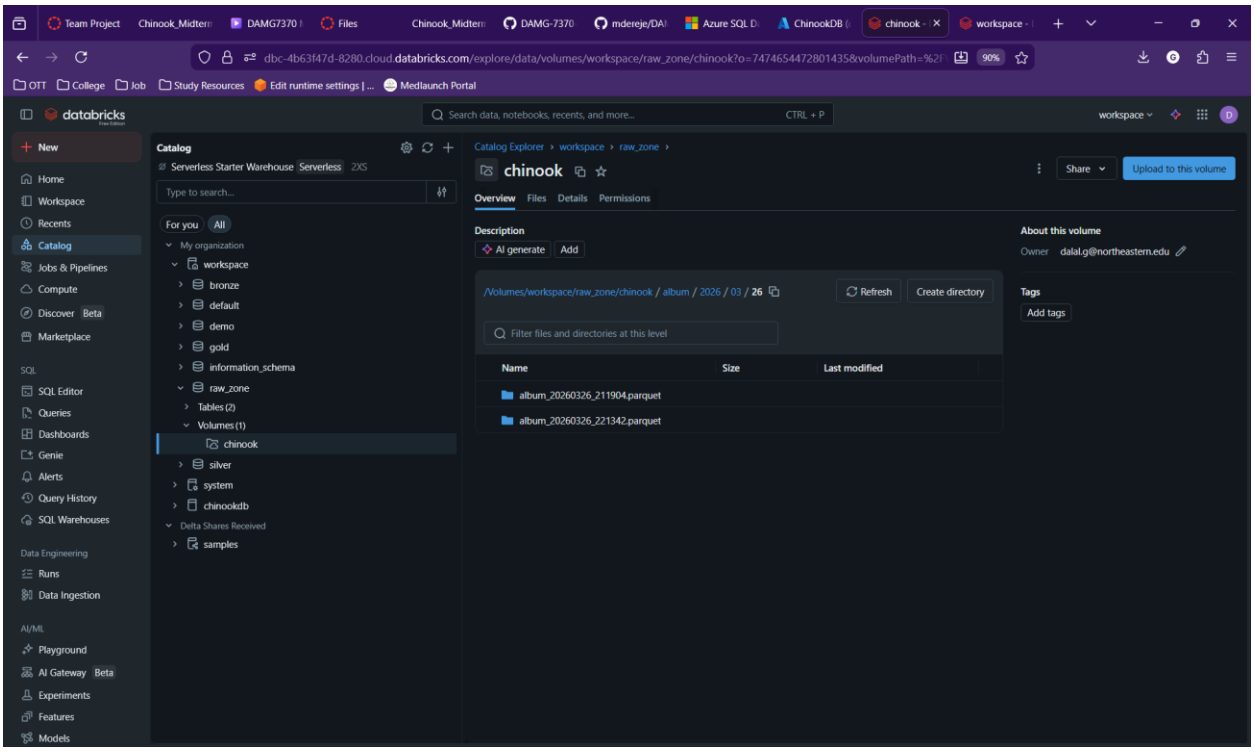
5.2 Raw Zone Load (02_load_raw)

Once connectivity is confirmed, the raw load notebook reads each active table directly from the Azure SQL source and writes an immutable Parquet snapshot to the Chinook volume. The file paths are dynamically generated using the table name and a UTC timestamp, following the pattern:

/Volumes/workspace/raw_zone/chinook/<table_name>/YYYY/MM/DD/<table_name>_YYYYMMDD_HHMMSS.parquet

After writing each file, the notebook reads the Parquet back and compares the row count against the source count. A mismatch triggers a FAILED status. Every result is logged to the child execution metrics table. At the end, if any table failed, the notebook raises an exception so the Job stops before proceeding to Bronze.

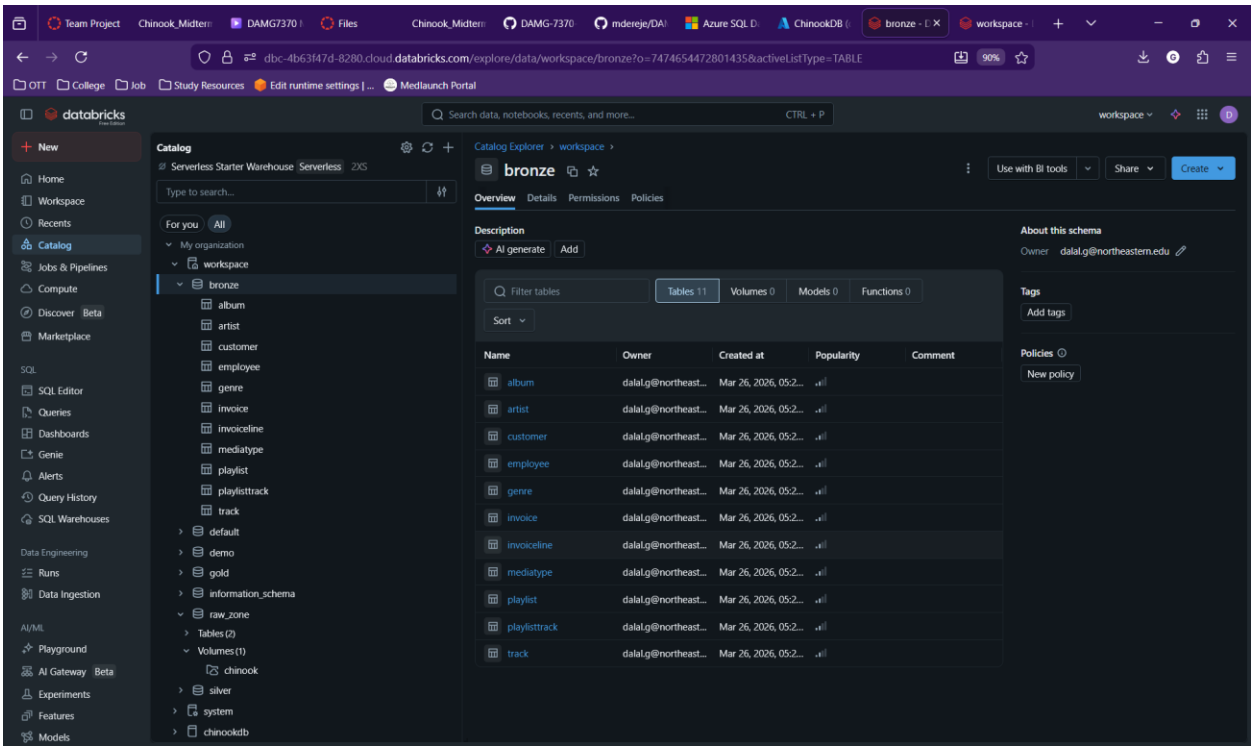
Using append mode with timestamped paths means previous snapshots are never overwritten. This is the key design principle of the Raw zone: it serves as a time-travel recovery layer. If something goes wrong downstream, we can always reprocess from any historical raw snapshot without touching the source system again.



5.3 Raw to Bronze (03_raw_to_bronze)

The Bronze layer is a straightforward Delta copy of the latest raw snapshot. This notebook queries the child execution metrics table, finds the most recent successful file location for each table (using max_by on execution_time), reads the Parquet from that path, and writes it to a Bronze Delta table in overwrite mode.

No transformations happen at this stage — Bronze is intentionally kept as an exact replica of Raw, just in Delta format. The benefit of Delta here is schema enforcement, time travel capabilities, and efficient reads for the next stage. Overwrite mode means Bronze always reflects the latest load, not accumulated history (that's Raw's job).



5.4 Bronze to Silver with Data Quality (04_bronze_to_silver)

The Silver stage is where data quality enforcement and cleaning take place. This notebook implements a DQX-style validation engine using native PySpark rather than the Databricks Labs DQX library directly. We went with this approach because the DQX library had cluster compatibility issues — specifically, a pipelines.id CONFIG_NOT_AVAILABLE error on standard compute clusters. The validation logic follows the same contract as DQX: rules are defined per table, and records are split into a passing set and a quarantine set.

5.4.1 Data Profiling

Before running validation rules, each Bronze table is profiled. The profiling step calculates null counts per column, identifies duplicate rows, and prints a schema summary with data types. This gives us visibility into the raw state of the data before any filtering happens.

5.4.2 Quality Rule Definitions

Quality rules are defined as a Python dictionary, with one entry per table. Each rule specifies the column to check, the type of check (not_null, not_null_and_not_empty, or min_value), and the criticality level (error or warn). Error-level rules cause records to be quarantined, while warn-level rules are logged but do not block the record from passing through.

For example, the Invoice table has not_null checks on InvoiceId, CustomerId, and InvoiceDate at the error level, plus a min_value check on Total (must be >= 0) at the warn level. The Customer table has a not_null check on CustomerId (error) and a not_null_and_not_empty check on Email (warn).

5.4.3 Quarantine Strategy

We chose to implement a quarantine table approach for handling failed records. Records that fail any error-level rule are written to a silver.quarantine Delta table with columns for the table name, run timestamp, error details (a JSON array of which rules failed), and the full row data serialized as JSON. This lets us review and debug problematic records without losing them.

Records that only trigger warn-level rules still pass through to Silver, since warnings are informational and do not represent data integrity violations.

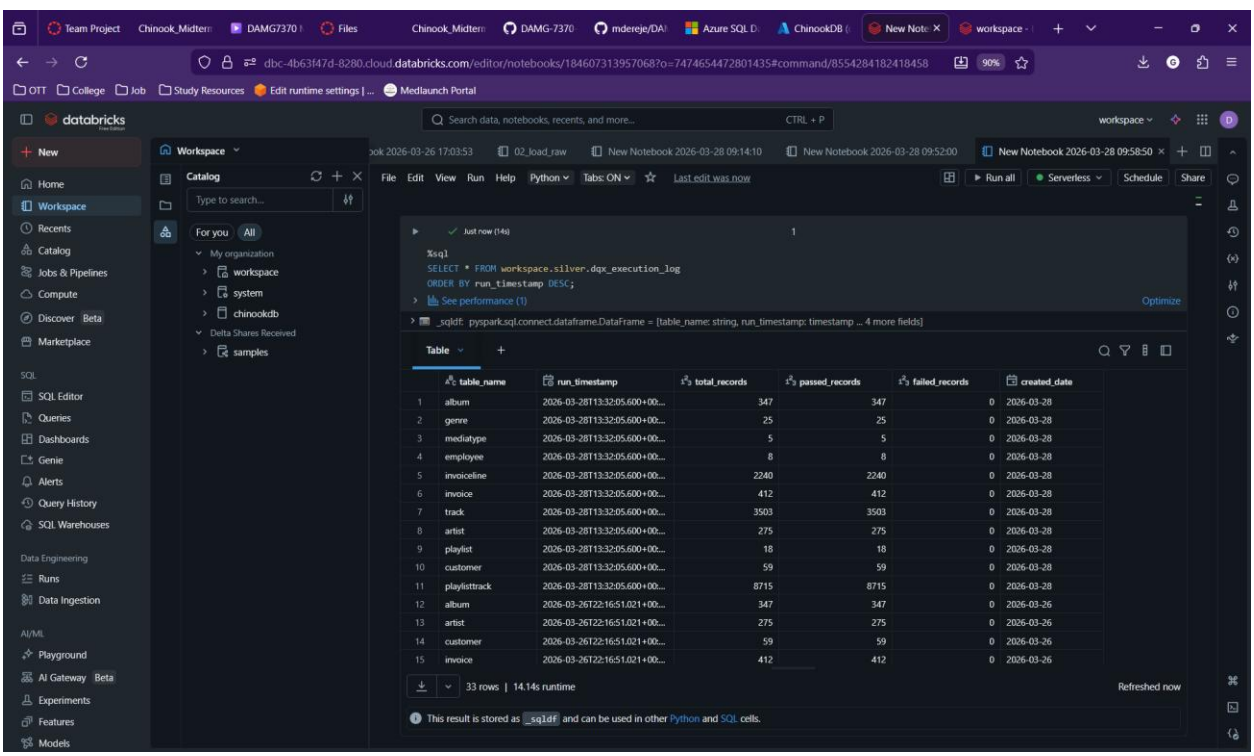
5.4.4 Silver Cleaning Transformations

After validation, passing records go through cleaning transformations. All string columns are trimmed across every table. Certain tables get additional treatment: the Customer table has emails lowercased, null Company fields defaulted to “Individual,” and null Fax fields defaulted to “N/A.” The Invoice table has InvoiceDate cast to a proper timestamp type, and the Employee table has BirthDate and HireDate similarly cast.

Silver tables are written in overwrite mode with overwriteSchema enabled, so schema evolution is handled automatically if the source schema changes between runs.

5.4.5 DQX Execution Logging

Every table processed in this stage generates a log entry in the silver.dqx_execution_log table. Each entry records the table name, run timestamp, total records, passed records, and failed records. This provides a historical view of data quality trends across pipeline runs.



6. Gold Layer — Dimensional Model

The Gold layer transforms the cleaned Silver tables into a star schema dimensional model. This includes eight dimension tables, one transaction-level fact table, and one aggregated fact table.

6.1 Dimension Tables

The following dimensions were built from Silver:

Dimension	Source	Key Columns	SCD Type
dim_artist	silver.artist	artist_key, artist_id, artist_name	Type 1
dim_genre	silver.genre	genre_key, genre_id, genre_name	Type 1
dim_mediatype	silver.mediatype	media_type_key, media_type_id, media_type_name	Type 1
dim_album	silver.album	album_key, album_id, album_title, artist_id	Type 1
dim_track	silver.track	track_key, track_id, track_name, album_id, genre_id, etc.	Type 1
dim_employee	silver.employee	employee_key, employee_id, employee_name, title	Type 1
dim_playlist	silver.playlist	playlist_key, playlist_id, playlist_name	Type 1
dim_date	silver.invoice	date_key (YYYYMMDD), full_date, year, quarter, etc.	Type 1

All Type 1 dimensions are written in overwrite mode with surrogate keys generated using monotonically_increasing_id(). The dim_date table is derived from distinct InvoiceDate values in the Silver layer and includes year, quarter, month, month name, day of month, day of week, day name, and a weekend flag.

6.2 dim_customer — SCD Type 2

The Customer dimension is the only table that implements SCD Type 2, meaning it tracks historical changes to customer attributes over time. The table includes a surrogate key (customer_key), the natural key (customer_id), all customer attributes, and three SCD columns: effective_start_date, effective_end_date (null for current records), and is_current (boolean flag).

On the first pipeline run, all incoming customer records are inserted as current rows with effective_start_date set to the run date and is_current = true. On subsequent runs, the notebook compares incoming Silver data against current Gold records across all tracked columns: FirstName, LastName, Company, Address, City, State, Country, PostalCode, Phone, Fax, Email, and SupportRepId.

If any column differs for a given customer_id, the existing row is expired (effective_end_date set to the current date, is_current flipped to false) and a new row is inserted with the updated attributes. Brand-

new customers that do not exist in Gold at all are simply inserted as new current rows. The surrogate key is assigned incrementally based on the current maximum key value in the table.

During our demo, loading Version 2 of the dataset produced new SCD Type 2 records for customers whose attributes changed between the two versions. We verified this by querying dim_customer and confirming that changed customers had both a historical row (is_current = false) and a current row (is_current = true).

6.3 fact_sales

The main fact table is built by joining Silver invoiceline and invoice tables with Gold dimension tables to resolve surrogate keys. Each row represents a single invoice line item and includes invoice_line_id, invoice_id, customer_key, track_key, date_key, support_rep_key, quantity, unit_price, line_total (quantity times unit_price), billing_country, and billing_city.

The joins are structured as: InvoiceLine joins Invoice on InvoiceId, then joins dim_customer (current records only) on CustomerId, dim_track on TrackId, dim_date on the date portion of InvoiceDate, and dim_employee on SupportRepId. Left joins are used for dimension lookups to avoid dropping invoice lines if a dimension key happens to be missing.

6.4 fact_sales_customer_agg

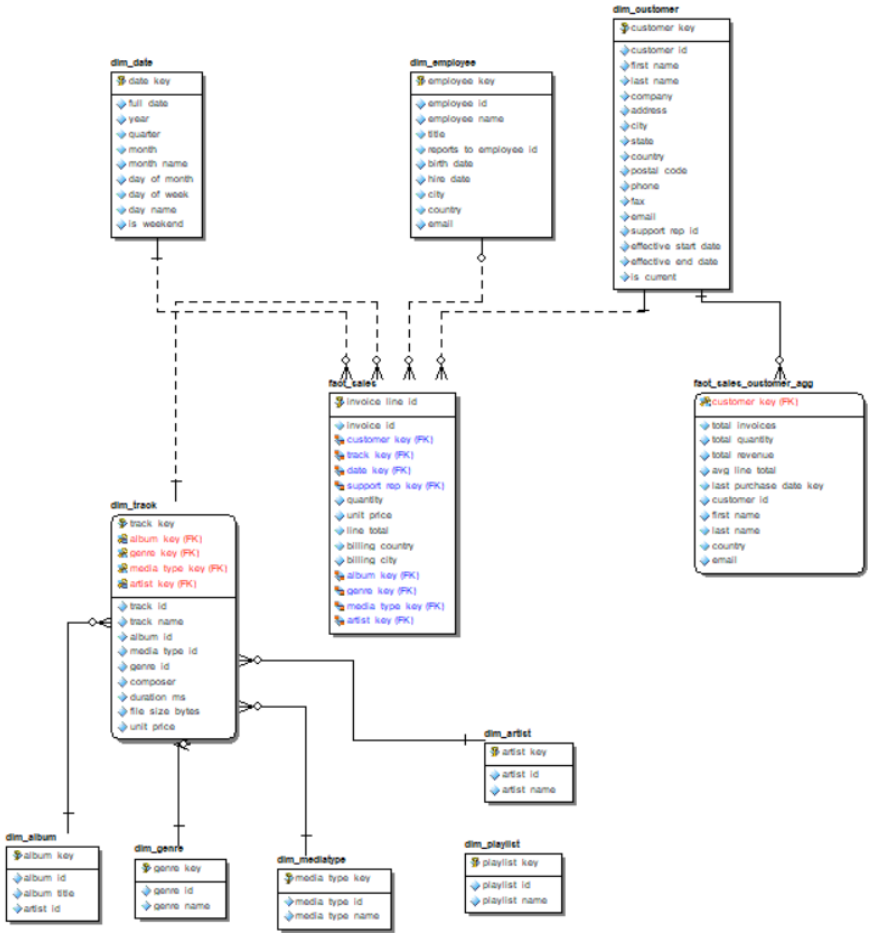
This aggregated fact table is built from fact_sales (not directly from Silver, as required by the project specification). It groups by customer_key and computes total invoices, total quantity, total revenue, average line total, and the last purchase date key. The result is joined back to dim_customer for identifying attributes like name, country, and email.

7. Gold Layer Schema Definition

In addition to the PySpark pipeline that creates the Gold tables programmatically, we produced a formal SQL DDL script for the star schema. This script is compatible with ER/Studio Data Architect for generating entity-relationship diagrams and uses ANSI SQL with SQL Server dialect conventions.

The DDL defines all primary keys, foreign key relationships between fact and dimension tables, and appropriate data types. The fact_sales table references dim_customer, dim_track, dim_date, and dim_employee through foreign keys. The dim_track table references dim_album, dim_genre, and dim_mediatype. And dim_album references dim_artist through artist_id.

This formal schema definition was created before writing the Gold notebook code, serving as the mapping document and blueprint for the dimensional model implementation.



8. Job Orchestration

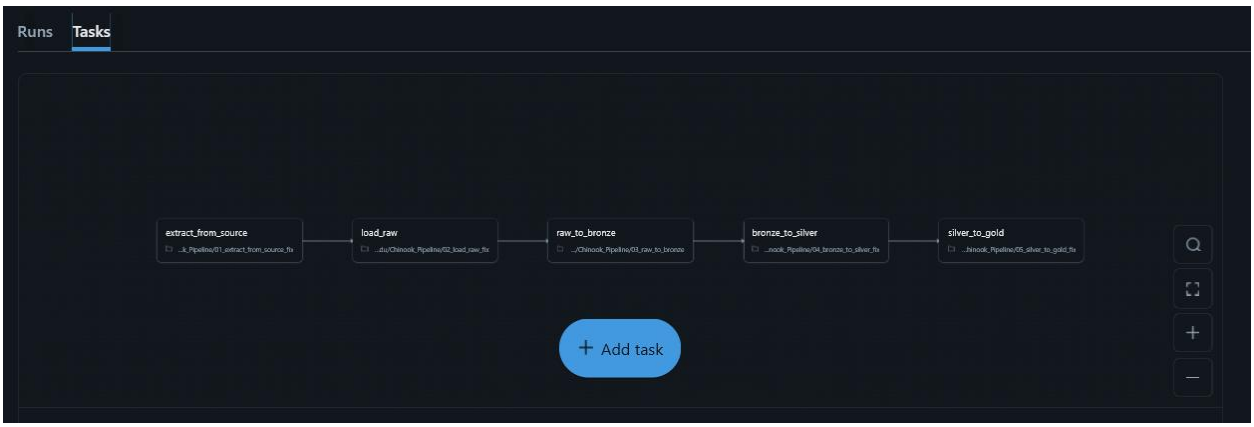
The full pipeline is orchestrated using a Databricks Job defined in a YAML configuration file (chinook_pipeline.yaml). Manual notebook execution is not used for final runs — the Job handles sequencing, parameter passing, and failure handling.

8.1 Task Configuration

The Job contains four tasks in a strict linear dependency chain:

Order	Task Key	Notebook	Depends On
1	load_raw	02_load_raw_fix	—
2	raw_to_bronze	03_raw_to_bronze	load_raw
3	bronze_to_silver	04_bronze_to_silver_fix	raw_to_bronze
4	silver_to_gold	05_silver_to_gold_fix	bronze_to_silver

Each task specifies its notebook path and passes required parameters through the base_parameters configuration. The pipeline can be repointed to a different environment by changing these parameter values in the Job definition.



8.2 Git Integration

The YAML configuration also sets up Git integration, pointing the Job to our GitHub repository with a chinook_pipeline tag. This means the Job pulls notebook code directly from version control, ensuring what runs in the pipeline matches what is committed in the repo. The Job is configured with performance-optimized compute and queue enabled for handling concurrent requests.

9. Testing and Verification

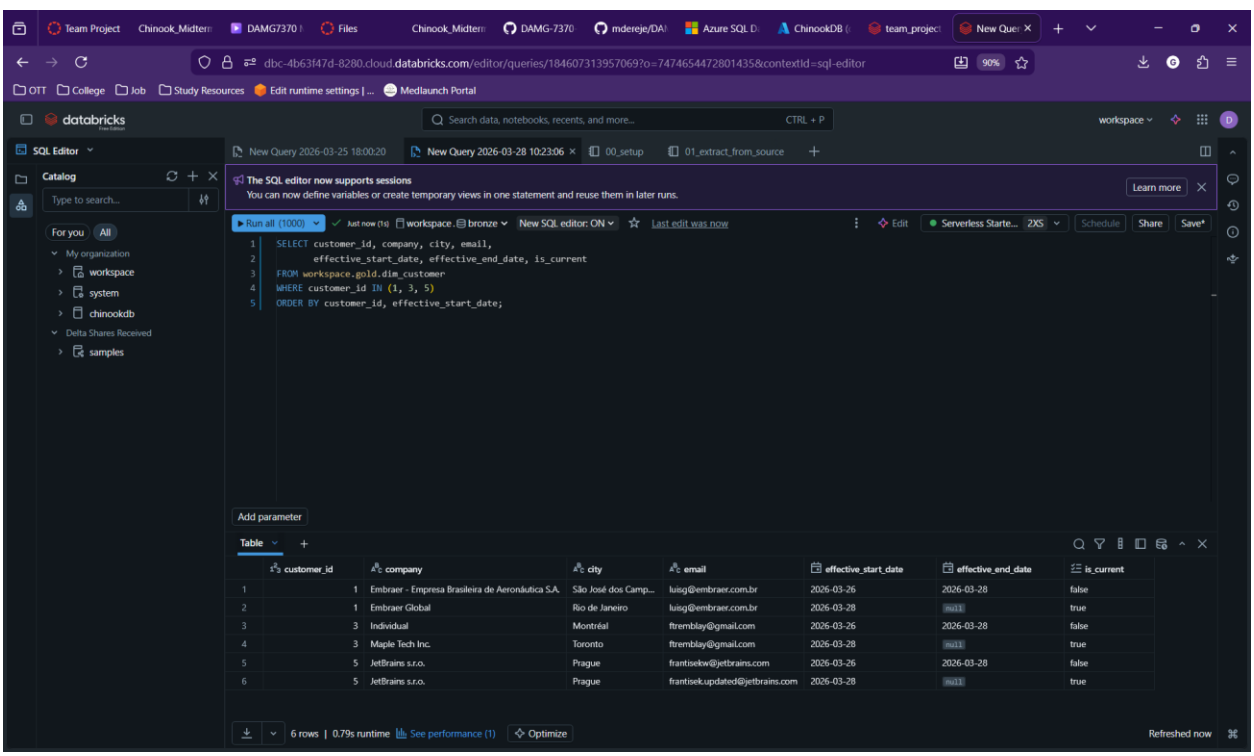
9.1 Version 1 Load

The first full pipeline run loaded all 11 Chinook tables from Version 1 of the dataset. We verified that every stage completed successfully by checking the pipeline_metadata_child table for all SUCCESS entries and confirming that source_row_count matched target_row_count for every table. The Gold layer was queried to confirm the dimensional model was correctly populated and surrogate keys were properly assigned.

9.2 Version 2 Load and SCD Type 2 Verification

When Version 2 was loaded, the pipeline detected changes in certain customer records. The SCD Type 2 logic correctly expired old rows (setting effective_end_date and is_current = false) and inserted new current rows with updated attributes. We verified this by querying dim_customer for specific changed customer IDs and confirming both historical and current versions existed with the correct effective dates.

During development, we encountered a bug in customer_key assignment where surrogate key values were not incrementing correctly across SCD2 merges. Pratik identified and fixed the issue by properly tracking the max existing key before inserting new versions. The corrected behavior was confirmed by querying dim_customer and verifying that all customer_key values were unique.



9.3 Data Quality Validation

We checked the silver.dqx_execution_log table to confirm that profiling and validation ran for all 11 tables. The quarantine table was also inspected to verify that records failing error-level rules were properly captured with their error details and original row data.